App Dev-1 Project Report

Student Details

Name: Pranjal Dey

Roll no. – 24F3000167

Email – 24f3000167@ds.study.iitm.ac.in

Project Details

Project Title**:** Placement Portal System

Problem Statement**:** To design and build a role-based web application that streamlines the campus placement process by connecting students, companies, and administrators on a single platform. The system manages company registrations, drive approvals, student applications, and status tracking in a secure and structured manner.

Approach**:** The application was developed using Flask with a modular Blueprint-based structure. SQLAlchemy was used for database modeling with SQLite as the backend database. Flask-Login was integrated for authentication and session protection.

The system supports three roles:

- Admin – Approves companies and drives, manages blacklist, monitors platform activity.
- Company – Creates and manages placement drives, reviews applications.
- Student – Applies for drives and tracks application status.

The frontend is built using Jinja2 templates and Bootstrap for responsive design.

AI/LLM Declaration

AI assistance (ChatGPT) was used approximately **28%** during the development of this project.

The AI was used for:

- Suggesting Flask route organization and Blueprint structure
- Assisting with HTML template formatting (Jinja + Bootstrap)
- Helping with UI of the project using Bootstrap
- Debugging guidance for common Flask/SQLAlchemy errors
- Integration of Flask-Login and security password hashing.

The following were implemented manually:

- Complete business logic and role-based access control
- Database relationship design and corrections
- Drive approval and status flow handling
- Blacklist logic implementation
- Duplicate application prevention logic
- Testing, debugging, and final integration

AI was used strictly as a support tool for suggestions and formatting. All architectural decisions and core logic implementation were performed independently.

Database Schema

Tables

User

- id (Primary Key)
- username
- password (hashed)
- role (admin/student/company)

Student

- id (Foreign Key → User.id)
- name
- email
- skills
- resume
- blacklisted (boolean)

Company

- id (Foreign Key → User.id)
- name
- hr_contact
- website
- approved (boolean)
- blacklisted (boolean)

Drive

- id (Primary Key)
- company_id (Foreign Key → Company.id)
- title
- description
- eligibility
- skills_required
- salary_range
- deadline

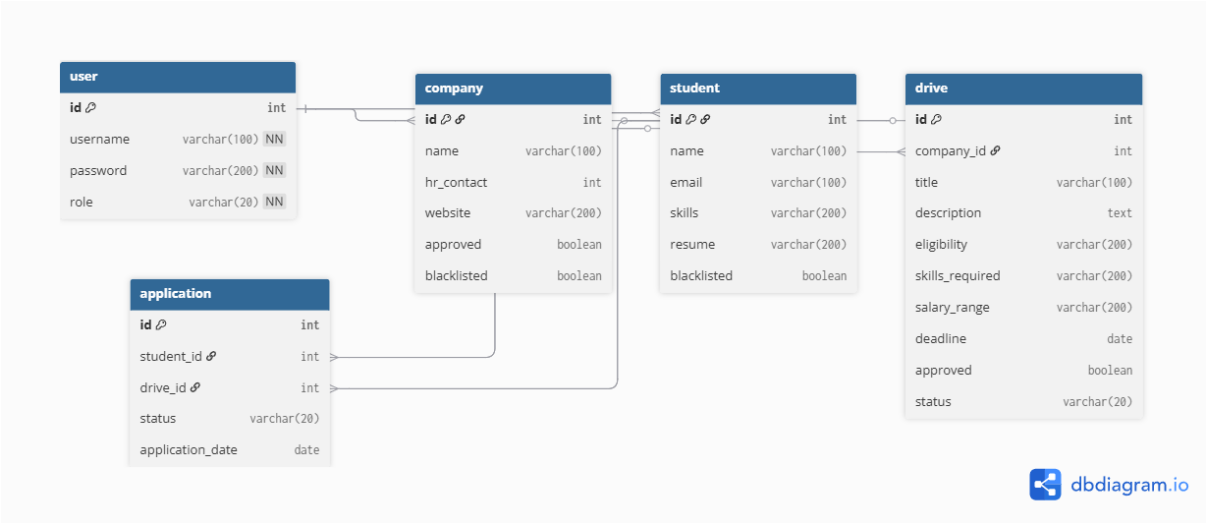
- approved (boolean)
- status (Pending / Active / Closed)

Application

- id (Primary Key)
- student_id (Foreign Key → Student.id)
- drive_id (Foreign Key → Drive.id)
- application_date
- status (Applied / Shortlisted / Selected / Rejected)

Relationships

- One-to-One → User → Student / Company
- One-to-Many → Company → Drive
- One-to-Many → Student → Application
- One-to-Many → Drive → Application



Technologies and Frameworks Used

Technology	Purpose
Flask	Core backend web framework

Technology	Purpose
SQLAlchemy	ORM for database operations
Jinja2	Template engine for rendering dynamic HTML
Bootstrap 5	Frontend styling and responsive UI
Flask-Login	Authentication and session management
Werkzeug	Password hashing and file handling
SQLite	Lightweight relational database

API Resource Endpoints

Endpoint	Method	Description
/login	POST	Authenticate user
/register/student	GET/POST	Student registration
/register/company	GET/POST	Company registration
/admin/dashboard	GET	Admin overview panel
/admin/approve_company/	GET	Approve company
/admin/approve_drive/	GET	Approve drive and activate
/company/dashboard	GET	Company dashboard
/company/create_drive	GET/POST	Create placement drive
/company/edit_drive/	GET/POST	Edit drive details
/company/drive/	GET	View drive applications
/company/update_status/<app_id>/	GET	Update application status
/student/dashboard	GET	Student dashboard
/student/apply/<drive_id>	GET	Apply to active drive
/student/profile	GET/POST	Edit student profile

Architecture and Features

Architecture Overview

- app.py – Application entry point

- models.py – SQLAlchemy database models
- routes.py – Blueprint containing all route definitions
- templates/ – Jinja2 HTML templates
- statics/uploads – Mainly for uploads of resume of students

Implemented Features

- Role-based authentication (Admin, Company, Student)
- Admin approval workflow for companies and drives
- Company profile management
- Drive creation and editing using shared form template
- Drive status management (Pending, Active, Closed)
- Student application system
- Application review and shortlisting system
- Resume upload functionality (PDF only)
- Blacklisting control by admin
- Dashboard segregation by role
- Secure password hashing using Werkzeug

Additional Features

- Status validation to prevent applying to closed drives
- Protection against duplicate applications
- Role-based access control for secure operations
- Dynamic filtering of active vs. closed drives
- Reusable form template for create/edit drive

Video Submission Link –

https://drive.google.com/file/d/1jflT1416YNy1I9XrCpcXnrAVxLpMiSPS/view?usp=sharing